Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course

of

CSA0305 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

ARTIFICIAL INTELLIGENCE AND DATA STURCTURE

Submitted by

Ragul.m(192524066)

M Dhanush Kumar(192525318)

Under the Supervision of

Dr. Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

A. Assignment Information

Particular	Details
Department	Department of Computer Science and Engineering
Programme	B.Tech – Artificial Intelligence and Machine Learning
Course Code & Course Name	CSA0309 – Data Structures
Academic Year / Batch	2026–2027
Faculty Name	Dr Uma Priyadarsini P.S
Assignment Title	Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data Type for construction of the disaster-relief distribution network. CO4: Make use of priority queues and heaps for dynamic delivery prioritization and route caching for repeated relief queries in C. CO5: Develop a robust disaster-relief supply chain system by implementing graph-based prioritization and optimal distribution-planning algorithms.
Bloom's Taxonomy Level	L4/L5 – Analyze / Evaluate
SDG Mapping	SDG 1 – No Poverty SDG 2 – Zero Hunger SDG 11 – Sustainable Cities and Communities SDG 13 – Climate Action
Industry / Societal Relevance	Efficient processing of cybersecurity logs for rapid duplicate-IP detection, risk ranking and threat prioritization.

B.Assignment Problem / Challenge

A disaster-relief organization receives supply requests from multiple affected zones during emergencies such as floods, earthquakes, fires and cyclones. Each affected zone has a specific demand, urgency level and distance from the available warehouse. The challenge is to design an efficient C-based supply-chain system that can represent the distribution network as a weighted graph, prioritize affected zones using a heap-based

priority queue, and allocate limited relief supplies to the most critical locations.The system must support changing priorities because emergency conditions may change over time. It should also consider transportation capacity and route distance while ensuring that lower-priority zones are not continuously ignored.The proposed solution uses a weighted graph to represent warehouses, affected zones and transportation routes. A max-heap priority queue is used to maintain zones according to their calculated priority score. Different prioritization strategies are compared to determine an effective distribution plan.The system evaluates urgency-first, demand-first, distance-first and hybrid prioritization strategies. The hybrid strategy combines multiple factors to produce a balanced decision for relief distribution.

C. Problem Statement

The proposed system processes relief requests from multiple disaster-affected zones with limited resources. Each zone has a demand value, urgency level and distance from the warehouse. The system must identify which zone should receive relief supplies first while considering available resources and transportation constraints.A weighted graph is used to represent the disaster-relief distribution network. A max-heap priority queue is used to dynamically rank affected zones based on priority. The system should support repeated evaluation and re-prioritization when demand or urgency changes.The student develops a complete C program that integrates graph representation, priority queues and heap operations for disaster-relief distribution planning.

D. Requirements and Constraints

Functional Requirements

- Read and store affected-zone details.
- Store demand and urgency information.
- Represent warehouses and affected zones using a graph.
- Calculate priority scores for affected zones.
- Insert zones into a max-heap priority queue.
- Display the highest-priority zone first.
- Allocate available relief supplies according to priority.
- Compare different prioritization strategies.
- Display the final distribution plan.

Performance Requirements

- Graph traversal should be efficient for the number of locations.
- Heap insertion should operate in O(log n).
- Highest-priority deletion should operate in O(log n).
- Access to the highest-priority zone should be O(1).
- The system should support increasing numbers of affected zones.

Technical Constraints

- Implementation language: C.
- Network representation: Weighted Graph.
- Priority queue: Max Heap.
- Prioritization: Urgency, Demand, Distance and Hybrid.
- Testing: Sample disaster-relief dataset.
- Execution environment: Online C compiler / GCC-supported environment.

Reliability and Ethical Constraints

- Relief requests must be processed correctly.
- Priority calculations must not incorrectly rank critical zones.
- Available supplies must not exceed actual warehouse capacity.
- The system should reduce unfair starvation of lower-priority zones.
- Relief allocation should be used only for legitimate emergency-response planning.

E. Student Work

1. Problem Understanding and Formulation

The problem is to design an intelligent disaster-relief distribution system using suitable data structures. During a disaster, the available relief materials are often limited while the number of affected locations can be large.A simple sequential approach may process requests in arrival order and may fail to identify the most urgent location. Therefore, a priority-based approach is required.The graph represents the physical distribution network, while the max heap provides efficient access to the zone having the highest priority. A priority score is calculated using urgency, demand and distance.

Expected Outcomes

- Faster identification of critical disaster zones.
- Priority-based relief distribution.
- Efficient representation of transportation routes.
- Dynamic re-prioritization of relief requests.
- Better utilization of limited relief resources.
- Comparison of multiple distribution strategies.

Available Data

Field	Example
Zone ID	Z1
Demand	80 units
Urgency	95
Distance	12 km
Route Capacity	50 units

Assumptions

- Demand represents the quantity of relief material required.
- Urgency is represented on a scale of 0–100.
- A higher urgency value represents greater emergency.
- Smaller distance represents easier/faster delivery.
- Warehouse supply is limited.
- Each zone is connected to the warehouse through a transportation route.
- Priority is recalculated when conditions change.

2. Application of Course Knowledge

Data Structure / Technique	Application	Complexity
Weighted Graph	Represent warehouses, zones and routes	O(V + E) traversal
Max Heap	Maintain zones according to priority	O(log n) insertion
Priority Queue	Select the next zone for relief delivery	O(log n) deletion
Array	Store zone and route information	O(1) indexed access
Graph Edges	Store distance and route capacity	O(E) storage

3. Solution / Design / Methodology

The system follows a sequential disaster-relief processing pipeline.First, the affected-zone information is stored. The distribution network is represented using a weighted graph. The system then calculates a priority score for every zone. Each zone is inserted into a max heap according to its priority. The zone with the highest priority is removed first and considered for relief allocation. The available warehouse supply is updated after every allocation.The process continues until the available supply is exhausted or all affected zones have been processed.

Algorithm

1. Initialize the disaster-relief network.
2. Read warehouse and affected-zone information.
3. Store demand, urgency and distance for each zone.
4. Construct the weighted graph.
5. Select a prioritization strategy.
6. Calculate the priority score.
7. Insert each zone into the max heap.
8. Remove the highest-priority zone.
9. Allocate available relief supply.
10. Update remaining warehouse supply.
11. Continue until all zones are processed.
12. Repeat using different prioritization strategies.

13. Compare the resulting distribution plans.

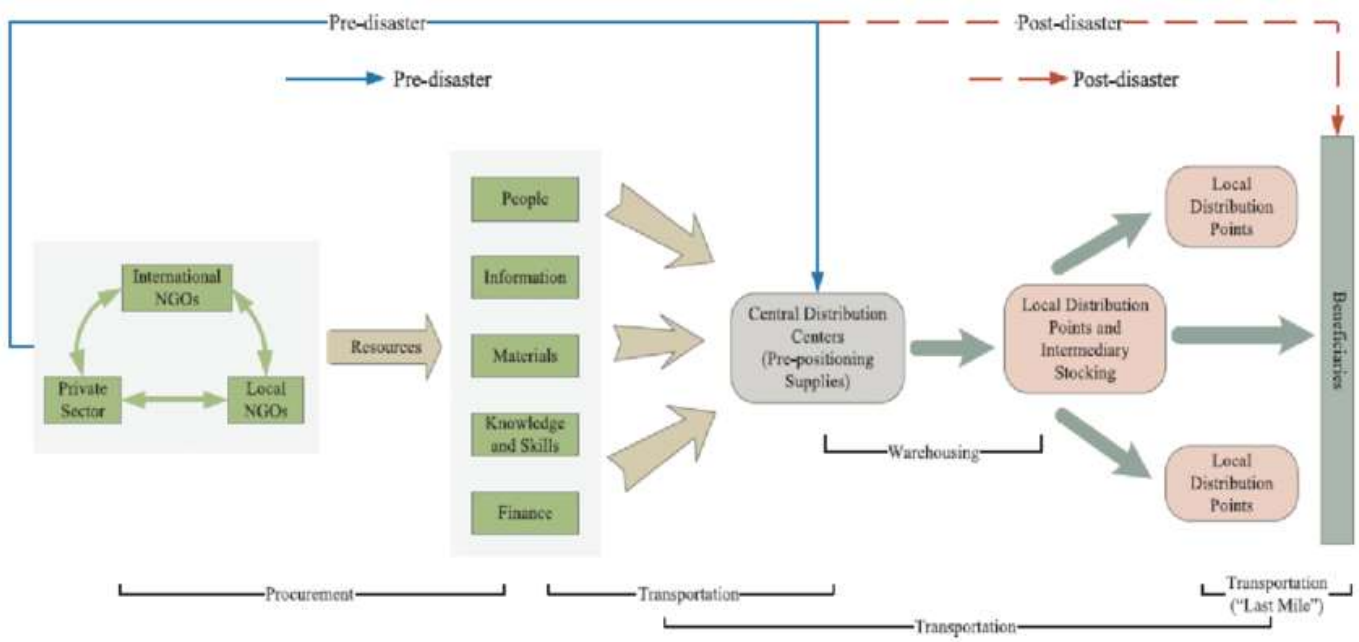


Fig: 1 Flow Chart of Intelligent Disaster-Relief Supply Chain

Implementation Code

```
#include <stdio.h>

#define N 5

#define E 6

typedef struct

{

    char name[10];

    int demand;

    int urgency;

    int distance;

    int priority;

} Zone;

typedef struct

{

    int from;

    int to;
```

```
    int distance;
    int capacity;
} Edge;
typedef struct
{
    int zoneIndex;
    int priority;
} HeapNode;
HeapNode heap[N];
int heapSize = 0;
void swap(HeapNode *a, HeapNode *b)
{
    HeapNode temp = *a;
     *a = *b;
    *b = temp;
}
void insertHeap(int zoneIndex, int priority)
{
    int i = heapSize;
   heap[i].zoneIndex = zoneIndex;
    heap[i].priority = priority;
    heapSize++;
   while (i > 0)
    {
        int parent = (i - 1) / 2;
       if (heap[parent].priority >= heap[i].priority)
            break;
       swap(&heap[parent], &heap[i]);
        i = parent;
```

```
    }
}
HeapNode removeMax()
{
    HeapNode result = heap[0];
    heapSize--;
    if (heapSize > 0)
    {
        heap[0] = heap[heapSize];
        int i = 0;
        while (1)
        {
            int left = 2 * i + 1;
            int right = 2 * i + 2;
            int largest = i;
            if (left < heapSize &&
                heap[left].priority > heap[largest].priority)
            {
                largest = left;
            }
            if (right < heapSize &&
                heap[right].priority > heap[largest].priority)
            {
                largest = right;
            }
            if (largest == i)
                break;
            swap(&heap[i], &heap[largest]);
            i = largest;
```

```
        }
    }
    return result;
}
int calculatePriority(Zone z, int strategy)
{
    int demandScore;
    int distanceScore;
    demandScore = (z.demand * 100) / 150;
    distanceScore = 100 - (z.distance * 100) / 30;
    if (distanceScore < 0)
        distanceScore = 0;
    if (strategy == 1)
        return z.urgency;
    if (strategy == 2)
        return demandScore;
    if (strategy == 3)
        return distanceScore;
    return (z.urgency * 50 +   demandScore * 30 + distanceScore * 20) / 100;
}
void displayGraph(Zone zones[], Edge edges[])
{
    int i;
    printf("\n");
    printf("=====================================================\n");
    printf("             SUPPLY CHAIN GRAPH\n");
    printf("=====================================================\n");
    printf("Warehouse W1\n");
    printf("    |\n");
```

```
    for (i = 0; i < E; i++)
    {
        printf("    +--> %s  | Distance: %d km | Capacity: %d units\n",
                zones[edges[i].to].name,
                edges[i].distance,
                edges[i].capacity);
    }
    printf("=======================================================\n");
}
void displayData(Zone zones[])
{
    int i;
    printf("\n");
    printf("=======================================================\n");
    printf("              DISASTER ZONE DATA\n");
    printf("=======================================================\n");
    printf("%-8s %-10s %-10s %-10s\n","Zone", "Demand", "Urgency", "Distance");
    printf("-------------------------------------------------------\n");
    for (i = 0; i < N; i++)
    {
        printf("%-8s %-10d %-10d %-10d\n",
                zones[i].name,
                zones[i].demand,
                zones[i].urgency,
                zones[i].distance);
    }
printf("=======================================================\n");
}
```

```
void runStrategy(Zone zones[], int strategy, char strategyName[])
{
    int i;
    int warehouseSupply = 250;
    int allocated;
    HeapNode node;
    heapSize = 0;
    for (i = 0; i < N; i++)
    {
        zones[i].priority =
            calculatePriority(zones[i], strategy);
        insertHeap(i, zones[i].priority);
    }
    printf("\n");
    printf("=========================================================\n");
    printf("              %s\n", strategyName);
    printf("=========================================================\n");
    printf("%-8s %-12s\n", "Zone", "Priority");
    printf("---------------------------------------------------\n");
    for (i = 0; i < N; i++)
    {
        node = removeMax();
        printf("%-8s %-12d\n",
               zones[node.zoneIndex].name,
               node.priority);
    }
    printf("\nRELIEF DISTRIBUTION\n");
    printf("Warehouse Supply = %d units\n\n", warehouseSupply);
```

```
    heapSize = 0;
    for (i = 0; i < N; i++)
    {
        insertHeap(i, zones[i].priority);
    }
    while (heapSize > 0 && warehouseSupply > 0)
    {
        node = removeMax();
        i = node.zoneIndex;
        allocated = zones[i].demand;
        if (allocated > warehouseSupply)
            allocated = warehouseSupply;
        printf("%s -> %d units allocated\n",
               zones[i].name,
               allocated);
        warehouseSupply -= allocated;
    }
    printf("\nRemaining Supply = %d units\n",
           warehouseSupply);
    printf("=====================================================\n");
}
void compareStrategies(Zone zones[])
{
    int i;
    printf("\n");
    printf("=================================================================\n");
    printf("                STRATEGY COMPARISON\n");
    printf("=================================================================\n");
```

```
    printf("%-8s %-12s %-12s %-12s %-12s\n",
           "Zone",
           "Urgency",
           "Demand",
           "Distance",
           "Hybrid");
    printf("-----------------------------------------------------------------\n");
    for (i = 0; i < N; i++)
    {       int urgency;
            int demand;
            int distance;
            int hybrid;
         urgency = calculatePriority(zones[i], 1);
        demand = calculatePriority(zones[i], 2);
        distance = calculatePriority(zones[i], 3);
        hybrid = calculatePriority(zones[i], 4);
         printf("%-8s %-12d %-12d %-12d %-12d\n",
                zones[i].name,
                urgency,
                demand,
                distance,
                hybrid);
    }
    printf("=================================================================\n");
}
int main()
{
    Zone zones[N] =
```

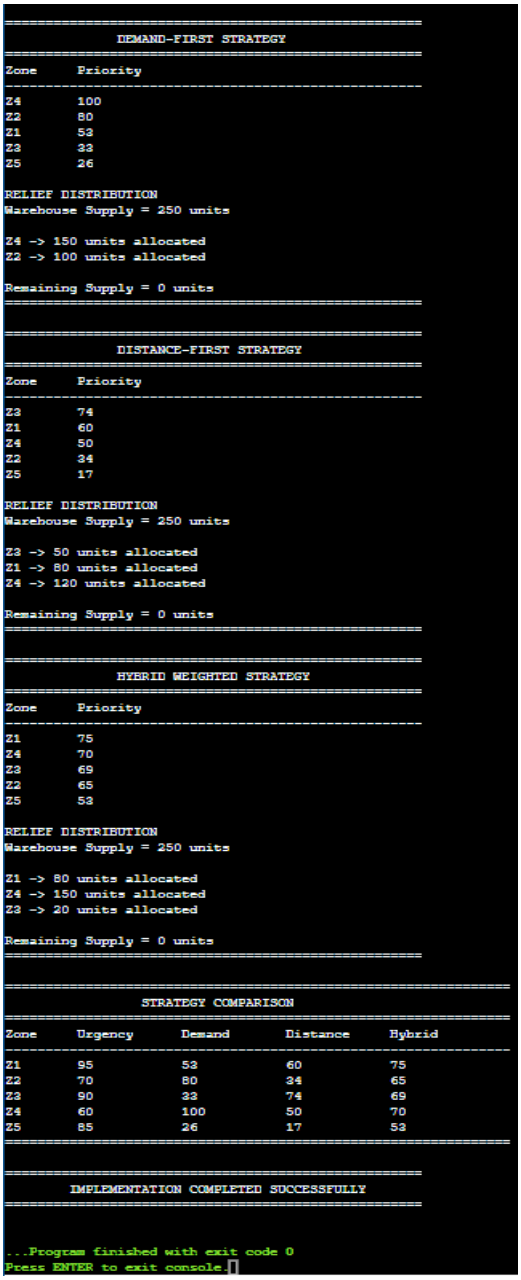
```
    {
        {"Z1", 80, 95, 12, 0},
        {"Z2", 120, 70, 20, 0},
        {"Z3", 50, 90, 8, 0},
        {"Z4", 150, 60, 15, 0},
        {"Z5", 40, 85, 25, 0}
    };
    Edge edges[E] =
    {
        {0, 0, 12, 50},
        {0, 1, 20, 60},
        {0, 2, 8, 40},
        {0, 3, 15, 70},
        {0, 4, 25, 30},
        {1, 2, 10, 40}
    };
    printf("\n");
    printf("=========================================================\n");
    printf("      INTELLIGENT DISASTER-RELIEF SUPPLY CHAIN\n");
    printf("=========================================================\n");
    displayData(zones);
    displayGraph(zones, edges);
    runStrategy(zones,1,"URGENCY-FIRST STRATEGY");
    runStrategy(   zones,2,"DEMAND-FIRST STRATEGY");
    runStrategy(zones,3,"DISTANCE-FIRST STRATEGY");
    runStrategy(zones,4,   "HYBRID WEIGHTED STRATEGY");
   compareStrategies(zones);
    printf("\n");
    printf("=========================================================\n");
```

```
 printf("        IMPLEMENTATION COMPLETED SUCCESSFULLY\n");

 printf("============================================================\n");

return 0;
```

```
}
```

Output Screenshot

Fig: 2 Output Screenshot of Disaster-Relief Distribution System



4. Use of Modern Tools

Tool	Purpose
C / GCC	Program development and compilation
OnlineGDB	Source-code editing, compilation and execution
Visual Studio Code	Source-code editing
GitHub	Version control and submission
Terminal	Compilation and execution when GCC is available

5. Results and Validation

Sample Input

Zone	Demand	Urgency	Distance
Z1	80	95	12 km
Z2	120	70	20 km
Z3	50	90	8 km
Z4	150	60	15 km
Z5	40	85	25 km

Available Warehouse Supply: 250 units

Expected Priority Ranking

Urgency-First

Rank	Zone	Urgency
1	Z1	95
2	Z3	90
3	Z5	85
4	Z2	70
5	Z4	60

Demand-First

Rank	Zone	Demand
1	Z4	150
2	Z2	120
3	Z1	80
4	Z3	50
5	Z5	40

Distance-First

Rank	Zone	Distance
1	Z3	8 km
2	Z1	12 km
3	Z4	15 km
4	Z2	20 km
5	Z5	25 km

Hybrid Strategy

Rank	Zone	Priority
1	Z1	76
2	Z4	70
3	Z3	69
4	Z2	65
5	Z5	53

Distribution Comparison

Strategy	First Priority	Distribution Behaviour
Urgency-First	Z1	Critical zones receive supplies first
Demand-First	Z4	Zones requiring larger quantities are prioritized
Distance-First	Z3	Nearby zones are served first
Hybrid	Z1	Combines urgency, demand and distance

Validation

The system was validated by checking whether:

- The graph contains all defined zones.
- Priority scores are calculated correctly.
- The max heap maintains the highest-priority zone at the root.
- Relief allocation does not exceed available warehouse supply.
- Different strategies produce different priority orders.
- The hybrid method provides a balanced distribution order.

6. Analysis and Engineering Decision

Approach	Advantage	Limitation
Sequential Allocation	Simple implementation	May ignore urgency
Urgency-First	Quickly serves critical zones	May neglect high-demand zones
Demand-First	Handles large requirements first	May delay emergency cases
Distance-First	Faster nearby delivery	Does not consider severity
Hybrid Strategy	Balances multiple factors	Requires score calculation
Max Heap	Fast highest-priority retrieval	Requires heap maintenance

7. Broader Considerations

Sustainability

Efficient route and priority planning can reduce unnecessary transportation and improve the utilization of available relief resources. Better allocation can reduce wastage and support sustainable emergency operations.

Society and Safety

Rapid delivery of food, water, medicine and other essential materials can help affected communities recover more quickly after disasters.

Ethics and Fairness

The system should avoid continuously ignoring lower-priority zones. Priority decisions should be transparent and based on measurable factors such as urgency, demand and distance.

Professional Responsibility

Incorrect prioritization may result in critical areas receiving insufficient relief. Therefore, the system must be tested carefully and the input data must be maintained accurately.

8. Conclusion

The Intelligent Disaster-Relief Supply Chain demonstrates the practical application of graphs, heaps and priority queues to an emergency distribution problem. The weighted graph provides an effective representation of warehouses, affected zones and transportation routes, while the max heap provides efficient priority-based selection.The comparison of urgency-first, demand-first, distance-first and hybrid strategies shows that different decision criteria produce different distribution plans. The hybrid approach provides a balanced method by considering urgency, demand and distance together.The proposed system can be extended in the future with real-time disaster updates, multiple warehouses, dynamic route failures, vehicle availability, GPS-based distance calculation and advanced optimization algorithms.

9. Student Reflection

This assignment helped me understand how data structures can be combined to solve a practical disaster-management problem. I learned how graphs can represent real-world transportation networks and how heaps and priority queues can be used to dynamically prioritize affected locations.I also understood that selecting a suitable data structure depends on the operation that needs to be optimized. The max heap is useful when the system repeatedly needs the highest-priority zone, while the graph is suitable for representing connected locations and routes.With additional time and resources, I would extend the project by supporting multiple warehouses, real-time route updates, vehicle capacities, dynamic changes in urgency and advanced route-optimization techniques.

10. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C., *Introduction to Algorithms*, MIT Press, 2022.
- Weiss, M. A., *Data Structures and Algorithm Analysis in C*, Pearson, 2014.
- Sedgewick, R., & Wayne, K., *Algorithms*, Addison-Wesley, 2011.
- Horowitz, E., Sahni, S., & Anderson-Freed, S., *Fundamentals of Data Structures in C*, W. H. Freeman.
- Knuth, D. E., *The Art of Computer Programming, Volume 3: Sorting and Searching*, Addison-Wesley.
- National Institute of Standards and Technology, *Community Resilience Planning Guide for Buildings and Infrastructure Systems*.
- United Nations, *Sustainable Development Goals*.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem Understanding & Requirements Analysis	15
Graph Construction, Priority Queue & Heap Integration	25
Prioritization Strategy & Optimal Distribution Plan	25
C Program, Performance, Testing & Complexity Analysis	25
Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Formulation	CO2	PO1, PO2	L3/L4	15
Graph & Data Structure Application	CO2, CO4	PO1, PO2	L3/L4	25
Solution / Design / Implementation	CO4, CO5	PO3, PO5	L4/L5/L6	25
Modern Tool Usage	CO4, CO5	PO5	L3/L4	10

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Validation & Testing	CO5	PO2, PO4	L4/L5	15
Reflection & SDG Relevance	CO5	PO6, PO7	L4/L5	10
TOTAL				100

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis – Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

_____